Université de Caen Normandie

Département Informatique

Rapport de Projet

Livre Dont Vous Êtes Le Héros

Analyse et modélisation par graphes

Réalisé par : Tounkara Fatoumata Françoise

Ouattara Gnine Asmaou

Kotin Olamide Naomie Syntiche

Khadir Tassadit

Niveau : Licence 2 Informatique

Encadrant : Grégory Bonnet

Date : 20 avril 2026

Année universitaire 2025–2026

Table des matières

1 Introduction **3**
1.1 Description générale du projet 3
1.2 Identification de la problématique 3
1.3 Présentation du plan du rapport 4

2 Structuration du projet **5**
2.1 Besoins du projet 5
2.2 Fonctionnalités implémentées 5
2.3 Organisation du projet 6

3 Éléments techniques **7**
3.1 Structures de données non triviales 7
3.1.1 Le graphe orienté 7
3.1.2 La pile pour l'exploration DFS 7
3.1.3 La file pour le parcours BFS 8
3.1.4 Le dictionnaire des prédécesseurs 8
3.2 Algorithmes non triviaux 8
3.2.1 Algorithme 1 : Exploration exhaustive (DFS) 8
3.2.2 Algorithme 2 : Plus court chemin (BFS) 9
3.2.3 Algorithme 3 : Simulation probabiliste (Monte Carlo) 10

4 Architecture du projet **12**
4.1 Diagramme des modules et des classes 12
4.2 Description des paquetages non standards utilisés 13
4.2.1 GraphStream (org.graphstream) 13
4.2.2 Java Collections Framework (java.util.*) 13
4.3 Chaînes de traitement 13
4.3.1 Description détaillée de la chaîne 14

5 Expérimentations et usages **15**
5.1 Cas d'utilisation 15
5.1.1 Cas 1 : Analyse d'un livre simple 15
5.1.2 Cas 2 : Calcul du plus court chemin 16
5.1.3 Cas 3 : Simulation probabiliste 16
5.2 Résultats quantifiables 16
5.2.1 Temps d'exécution des algorithmes 16
5.2.2 Nombre de chemins trouvés 16
5.2.3 Comparaison entre livres 16
5.2.4 Interprétation des résultats 17

6 Conclusion **18**

1. Introduction

1.1. Description générale du projet

Tout au long de notre parcours en licence informatique, nous avons manipulé de nombreuses structures de données : listes, piles, files, arbres. Pourtant, une structure fondamentale semble progressivement disparaître des maquettes pédagogiques : le graphe. Les cursus actuels abordent les graphes de manière souvent superficielle, parfois reléguée en fin de semestre ou traitée rapidement. Cette situation est d'autant plus paradoxale que les graphes sont omniprésents dans le monde contemporain. Les réseaux sociaux modélisent les relations entre utilisateurs sous forme de graphes. Les moteurs de recherche comme Google utilisent PageRank — un algorithme de parcours de graphe — pour classer les pages web. Les applications de navigation calculent des plus courts chemins. Les systèmes de recommandation s'appuient sur des graphes de similarités.

Face à ce constat, nous avons souhaité choisir un projet qui remette les graphes au centre du travail. Le sujet proposé — l'analyse des livres dont vous êtes le héros — nous est immédiatement apparu comme une opportunité idéale. En effet, ces ouvrages possèdent une structure naturellement non linéaire : le lecteur progresse dans l'histoire en faisant des choix, chaque choix le conduisant vers un nouveau paragraphe. Cette organisation correspond exactement à la définition d'un graphe orienté, où les paragraphes sont des sommets et les choix des arêtes.

Notre objectif principal est donc de concevoir un système capable de transformer un livre texte brut en une structure de graphe exploitable, puis d'analyser ce graphe pour en extraire des propriétés significatives. Au-delà de l'aspect technique, ce projet vise à démontrer la puissance des modèles graphes pour analyser des œuvres littéraires complexes.

1.2. Identification de la problématique

L'analyse des livres dont vous êtes le héros soulève plusieurs difficultés spécifiques qu'il convient d'identifier clairement.

Première difficulté : l'absence de structure explicite. Les livres sont fournis sous forme de fichiers texte brut. Contrairement à des formats comme XML ou JSON, aucun marqueur n'indique où commence un paragraphe, où se trouvent les choix, ou quelles sont les relations entre les éléments. Tout repose sur le formatage visuel (sauts de ligne,

numéros en début de ligne) et sur des formulations en langage naturel.

Deuxième difficulté : la diversité des formulations. Les choix ne sont pas toujours explicites. Dans certains livres, la phrase indiquant le choix peut être noyée dans le texte narratif. Les formulations varient : « rendez-vous au », « reportez-vous au », « allez à la page », etc. Notre analyseur doit être capable de reconnaître un maximum de variantes.

Troisième difficulté : les cas particuliers. Certains paragraphes contiennent des conditions (« Si vous possédez l'épée... »), des combats avec des monstres, des modifications de statistiques, ou des références circulaires créant des boucles. Certaines références peuvent même être invalides (numéros de paragraphe inexistants).

Quatrième difficulté : la taille du graphe. Pour un livre de taille moyenne (plusieurs centaines de paragraphes), l'exploration exhaustive peut générer des centaines de milliers de chemins. Les algorithmes doivent donc être efficaces.

1.3. Présentation du plan du rapport

Ce rapport s'organise en six chapitres. Le chapitre 2 présente la structuration du projet : les besoins identifiés, les fonctionnalités implémentées et l'organisation du travail au sein de l'équipe. Le chapitre 3 détaille les éléments techniques, c'est-à-dire les structures de données et les algorithmes non triviaux que nous avons développés. Le chapitre 4 décrit l'architecture du projet à travers des diagrammes de classes et de modules, ainsi que les chaînes de traitement. Le chapitre 5 présente les expérimentations menées et leurs résultats quantifiables. Enfin, le chapitre 6 conclut ce rapport par un récapitulatif des fonctionnalités et des propositions d'améliorations.

2. Structuration du projet

2.1. Besoins du projet

Avant toute réalisation technique, nous avons identifié quatre besoins fondamentaux.

Lecture et analyse du fichier texte. Le système doit lire un fichier texte brut et en extraire la structure cachée : détection automatique des débuts de paragraphe (lignes commençant par un nombre), association du texte narratif à chaque paragraphe, extraction des choix (description et destination), et détection des objets, modifications de statistiques et monstres éventuels.

Construction du graphe. Une fois les paragraphes et les choix extraits, le système construit un graphe orienté en deux phases : création des sommets (un sommet par paragraphe), puis création des arcs (un arc par choix reliant la source à la destination).

Analyse structurelle. Le système doit proposer plusieurs types d'analyse : exploration exhaustive de tous les chemins, calcul du plus court chemin entre deux paragraphes, évaluation probabiliste de la difficulté par simulation aléatoire, et calcul d'indicateurs globaux (nombre de paragraphes, densité, degré moyen, présence de cycles).

Visualisation. Enfin, le système doit offrir une représentation visuelle du graphe obtenu pour permettre une compréhension intuitive de sa structure.

2.2. Fonctionnalités implémentées

En réponse à ces besoins, nous avons développé sept fonctionnalités principales.

Un **analyseur syntaxique** parcourt le fichier ligne par ligne et extrait les paragraphes, choix, objets, modifications de statistiques et monstres. Il gère les cas particuliers : références invalides (ignorées avec alerte), cycles (non bloqués), et variations de mise en forme.

À partir des données extraites, le système **construit un graphe orienté** où chaque sommet contient une référence vers l'objet Paragraphe correspondant, et chaque arc représente un choix.

Un algorithme de **parcours en profondeur (DFS)** explore récursivement tous les chemins possibles, en évitant les boucles par vérification d'appartenance au chemin courant. Les chemins menant à une feuille (paragraphe sans choix) sont enregistrés et affichés.

Un algorithme de **parcours en largeur (BFS)** calcule le chemin le plus court entre deux paragraphes, garantissant un résultat optimal en nombre d'arcs.

Une **méthode de Monte Carlo** simule un grand nombre de parcours aléatoires pour estimer la probabilité qu'un lecteur choisissant au hasard atteigne une fin victorieuse.

La bibliothèque **GraphStream** génère une visualisation interactive du graphe, avec placement automatique des sommets par algorithme de type force-directed (Fruchterman-Reingold).

Enfin, le système calcule pour chaque livre un ensemble d'**indicateurs comparatifs** (nombre de paragraphes, densité, degré moyen, nombre de chemins victorieux, présence de cycles) et les présente dans un tableau.

2.3. Organisation du projet

Notre équipe de quatre membres s'est organisée dès le début avec une répartition claire des responsabilités, résumée dans le tableau 2.1.

Membre	**Tâches principales**
Tounkara Fatoumata Françoise	LivreHero,LivreHeroIterator,rédaction du beamer
Ouattara Gnine Asmaou	GraphLVDH,NoeudGraphe,AnalyseGraph, rédaction du rapport
Kotin Olamide Naomie Syntiche	Joueur,Inventaire,AffichageGraphFR,Main
Khadir Tassadit	Paragraphe, choix,Objet,Monstre

Table 2.1 – Répartition des tâches

Durant les séances encadrées, nous avons travaillé principalement sur le poste de Kotin Naomie. Ce choix nous a permis à tous d'avoir une vision complète du code et d'acquérir une bonne maîtrise de l'ensemble du projet. Chaque membre a ainsi pu comprendre toutes les parties, et non seulement sa propre contribution.

En fin de projet, nous nous sommes réparti la rédaction du rapport et du beamer

Le projet s'est déroulé en sept phases : analyse (1 semaine), prototypage (1 semaine), analyseur complet (2 semaines), construction du graphe (1 semaine), visualisation (quelques jours), tests (1 semaine), et rédaction du rapport (en parallèle).

3. Éléments techniques

3.1. Structures de données non triviales

3.1.1. Le graphe orienté

La structure centrale de notre projet est le graphe orienté. Formellement, un graphe orienté G est défini par un couple (V, E) où V est un ensemble de sommets et $E \subseteq V \times V$ un ensemble d'arcs (couples ordonnés de sommets). Dans notre implémentation, chaque sommet contient une référence vers un objet `Paragraphe`, et chaque arc est matérialisé par une liste d'adjacence.

Structure : Graphe orienté par listes d'adjacence

Représentation :

- Un dictionnaire (table de hachage) associant à chaque identifiant de paragraphe un objet `NoeudGraphe`.
- Chaque `NoeudGraphe` contient :
  - une référence vers l'objet `Paragraphe` correspondant ;
  - une liste (ArrayList) de références vers les `NoeudGraphe` voisins (accessibles par un choix).

Justification du choix :

- Accès en temps constant $\mathcal{O}(1)$ à un nœud par son identifiant.
- Parcours efficace des voisins d'un nœud.
- Complexité mémoire $\mathcal{O}(n + m)$ où n est le nombre de sommets et m le nombre d'arcs.

3.1.2. La pile pour l'exploration DFS

Pour l'exploration récursive des chemins (DFS), nous utilisons une pile (Stack) qui mémorise le chemin courant. Cette structure est essentielle pour le retour arrière (backtracking). À chaque appel récursif, le sommet courant est empilé ; à la sortie de l'appel, il est dépilé. Cette mécanique permet de maintenir à chaque instant le chemin exact depuis la racine jusqu'au nœud courant.

3.1.3. La file pour le parcours BFS

Pour le calcul du plus court chemin (BFS), nous utilisons une file (Queue) qui garantit un traitement dans l'ordre d'arrivée (FIFO : First In, First Out). Cette propriété est fondamentale pour explorer les nœuds niveau par niveau.

3.1.4. Le dictionnaire des prédécesseurs

Lors du parcours BFS, nous maintenons un dictionnaire associant à chaque nœud visité son prédécesseur (le nœud par lequel on l'a découvert pour la première fois). Ce dictionnaire permet de reconstruire le chemin une fois la destination atteinte, en remontant les prédécesseurs jusqu'à la source.

3.2. Algorithmes non triviaux

3.2.1. Algorithme 1 : Exploration exhaustive (DFS)

Principe

L'algorithme de parcours en profondeur explore récursivement tous les chemins possibles. Pour éviter les boucles infinies dans les graphes cycliques, on vérifie à chaque étape que le voisin considéré n'est pas déjà présent dans le chemin courant.

Pseudo-algorithme

Algorithme DFS : Exploration exhaustive des chemins

Entrée : sommet de départ d, graphe $G = (V, E)$
Sortie : affichage de tous les chemins menant à une feuille

Fonction DFS(sommet s, pile *chemin*) :

1. *chemin*.empiler(s)
2. **si** s n'a pas de voisin (feuille) **alors**
   (a) Afficher *chemin* (chemin complet trouvé)
3. **sinon**
   (a) **pour chaque** voisin v de s **faire**
      i. **si** v n'est pas dans *chemin* **alors**
         A. DFS(v, *chemin*)
4. *chemin*.dépiler()

Appel initial : DFS(d, pile vide)

Exemple d'exécution

Considérons un petit graphe à trois sommets : $1 \to 2$, $1 \to 3$, et 3 est une feuille.

1. Appel DFS(1, []) : on empile $1 \to$ chemin = [1]
2. 1 a deux voisins : 2 et 3
3. Premier voisin 2 : appel DFS(2, [1]) $\to$ 2 n'a pas de voisin $\to$ affichage [1,2]
4. Retour à 1, second voisin 3 : appel DFS(3, [1]) $\to$ 3 est une feuille $\to$ affichage [1,3]
5. Fin : on a trouvé deux chemins.

3.2.2. Algorithme 2 : Plus court chemin (BFS)

Principe

Le parcours en largeur explore tous les nœuds à distance k avant de passer à la distance $k + 1$. La première fois qu'on atteint le nœud d'arrivée, le chemin emprunté est nécessairement le plus court.

Pseudo-algorithme

Algorithme BFS : Plus court chemin

Entrée : sommet de départ d, sommet d'arrivée a, graphe $G = (V, E)$
Sortie : liste des sommets constituant le plus court chemin de d à a

1. Créer une file vide $file$
2. Créer un dictionnaire $pred$ (prédécesseurs) vide
3. Créer un ensemble $visites$ vide
4. Ajouter d à $file$ et à $visites$
5. **tant que** $file$ n'est pas vide **faire**
   (a) $courant \leftarrow file$.défiler()
   (b) **si** $courant = a$ **alors** sortir de la boucle
   (c) **pour chaque** voisin v de $courant$ **faire**
      i. **si** $v \notin visites$ **alors**
         A. Ajouter v à $file$ et à $visites$
         B. $pred[v] \leftarrow courant$
6. Reconstruire le chemin en partant de a et en remontant les prédécesseurs
7. Inverser l'ordre et retourner le chemin

Exemple d'exécution

Prenons un graphe : $1 \to 2$, $1 \to 3$, $2 \to 4$, $3 \to 4$. On cherche le plus court chemin de 1 à 4.

1. Initialisation : file = [1], visites = $\{1\}$
2. On défiler 1 : ses voisins sont 2 et 3. On ajoute 2 et 3 à file (file = [2,3]), avec pred[2]=1, pred[3]=1.
3. On défiler 2 : son voisin est 4. 4 n'est pas visité, on l'ajoute à file (file = [3,4]), pred[4]=2.
4. On défiler 3 : son voisin est 4, déjà visité (donc ignoré).
5. On défiler 4 : courant = 4 = arrivée $\to$ arrêt.
6. Reconstruction : on remonte depuis 4 : 4 $\to$ pred[4]=2 $\to$ pred[2]=1 $\to$ chemin [1,2,4].

3.2.3. Algorithme 3 : Simulation probabiliste (Monte Carlo)

Principe

La méthode de Monte Carlo estime une probabilité par répétition d'expériences aléatoires. Ici, on simule un grand nombre de parties où le lecteur choisit aléatoirement parmi les options disponibles à chaque étape. La proportion de parties victorieuses converge vers la vraie probabilité de succès.

Pseudo-algorithme

Algorithme : Simulation Monte Carlo

Entrée : sommet de départ d, nombre de simulations N, graphe $G = (V, E)$
Sortie : estimation de la probabilité de victoire

1. $\mathit{victoires} \leftarrow 0$
2. **pour** $i = 1$ à N **faire**
   (a) $\mathit{courant} \leftarrow d$
   (b) **tant que** *courant* a au moins un voisin **faire**
      i. Obtenir la liste des voisins de *courant*
      ii. Choisir aléatoirement un voisin v dans cette liste
      iii. $\mathit{courant} \leftarrow v$
   (c) **si** *courant* est une feuille de victoire **alors**
      i. $\mathit{victoires} \leftarrow \mathit{victoires} + 1$
3. **retourner** $\mathit{victoires}/N$

Exemple d'exécution

Pour un livre avec 50% de chances de victoire (une branche sur deux mène à la victoire), avec $N = 1000$ simulations, on obtiendra un résultat proche de 0,5. La loi des grands nombres garantit que l'erreur diminue comme $1/\sqrt{N}$.

4. Architecture du projet

4.1. Diagramme des modules et des classes

L'architecture de notre projet suit une séparation en trois couches distinctes : couche métier, couche de construction, et couche d'analyse. La figure **??** présente le diagramme de classes complet.

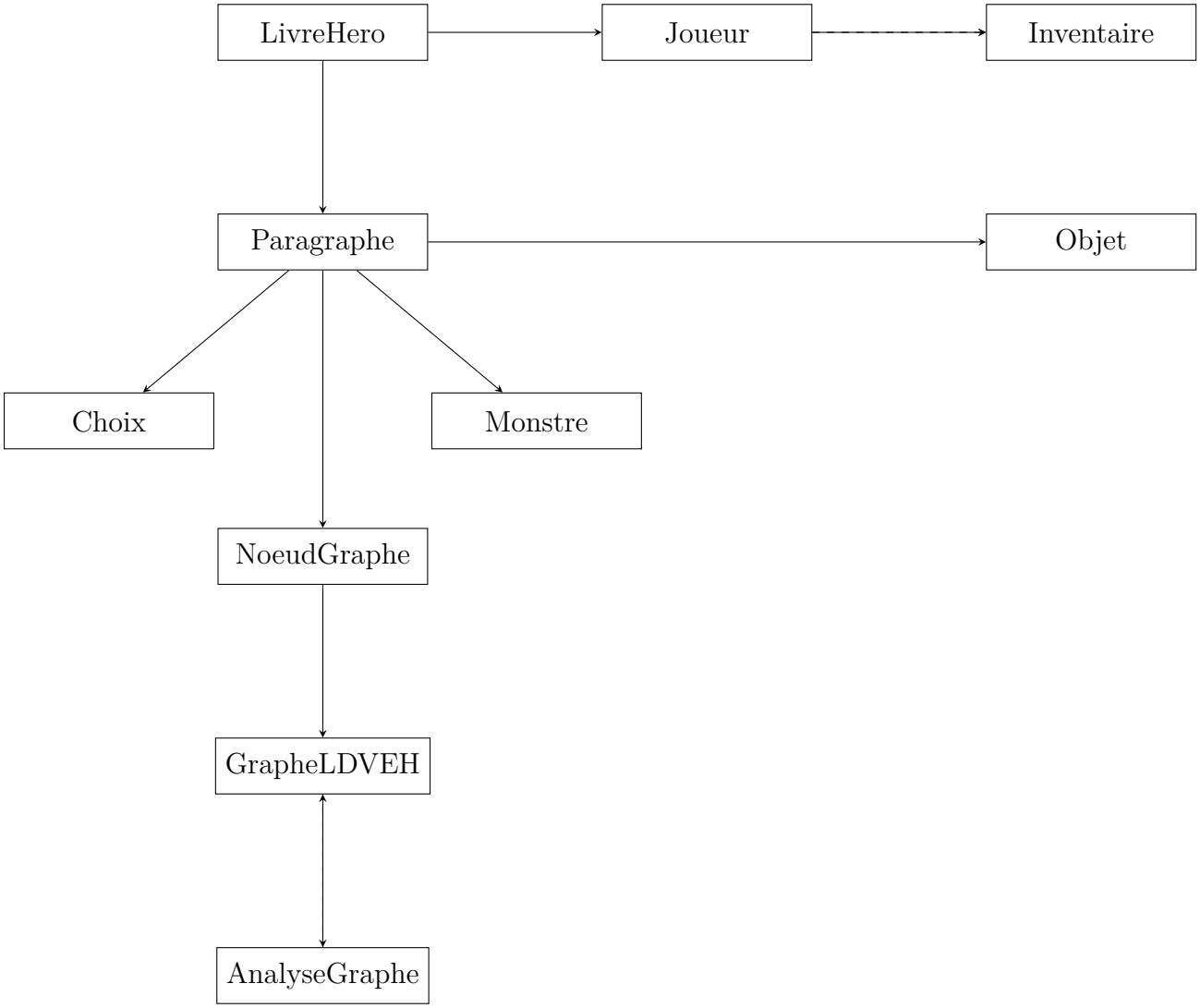

FIGURE 4.1 – Diagramme de classes UML de l'architecture logicielle

4.2. Description des paquetages non standards utilisés

4.2.1. GraphStream (org.graphstream)

GraphStream est une bibliothèque Java spécialisée dans la manipulation et la visualisation de graphes. Nous l'avons utilisée pour la visualisation (classe `AffichageGrapheFR`).

— **Apport dans le projet :** GraphStream nous a évité d'avoir à implémenter nous-mêmes un algorithme de placement de graphe. La bibliothèque fournit nativement l'algorithme de Fruchterman-Reingold (force-directed), qui produit des visualisations esthétiques et lisibles.

— **Justification de l'utilisation :** Sans cette bibliothèque, nous aurions dû calculer nous-mêmes les positions des sommets, ce qui est un problème complexe (NP-difficile dans sa version optimale). GraphStream offre une solution clé en main, bien documentée et performante.

4.2.2. Java Collections Framework (java.util.*)

Les collections Java standard (HashMap, ArrayList, LinkedList, Stack, Queue) sont utilisées intensivement.

— **HashMap :** pour stocker les paragraphes (accès par identifiant en $\mathcal{O}(1)$) et pour le dictionnaire des prédécesseurs dans BFS.

— **ArrayList :** pour les listes de voisins dans les nœuds du graphe.

— **Stack :** pour mémoriser le chemin courant dans DFS.

— **Queue (LinkedList) :** pour le parcours en largeur BFS.

4.3. Chaînes de traitement

La figure 4.2 illustre la chaîne de traitement complète, depuis le fichier texte brut jusqu'à l'affichage des résultats.

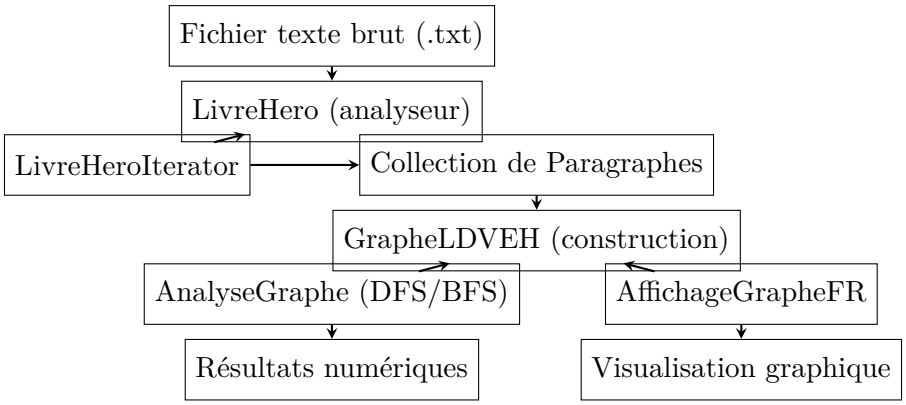

FIGURE 4.2 – Chaîne de traitement complète du projet

4.3.1. Description détaillée de la chaîne

1. **Entrée :** un fichier texte brut contenant un livre dont vous êtes le héros. Ce fichier respecte une mise en forme informelle : paragraphes numérotés, choix indiqués par des formulations comme « rendez-vous au », objets signalés par « vous trouvez », etc.
2. **Étape 1 : Analyse syntaxique (LivreHero).** La classe `LivreHero` parcourt le fichier ligne par ligne. Pour chaque ligne, elle vérifie si elle commence par un nombre (début de paragraphe). Si c'est le cas, elle crée un nouvel objet `Paragraphe`. Elle extrait également les choix, les objets, les modifications de statistiques et les monstres à l'aide d'expressions régulières.
3. **Étape 2 : Stockage (Collection de Paragraphes).** Les paragraphes sont stockés dans une HashMap associant chaque numéro à son objet `Paragraphe`. Cette structure permet un accès rapide lors de la construction du graphe.
4. **Étape 3 : Construction du graphe (GrapheLDVEH).** Le constructeur de `GrapheLDVEH` parcourt tous les paragraphes, crée un sommet pour chacun (objet `NoeudGraphe`), puis ajoute les arcs en fonction des choix.
5. **Étape 4a : Analyse (AnalyseGraphe).** La classe `AnalyseGraphe` implémente les algorithmes DFS (exploration exhaustive) et BFS (plus court chemin). Elle détecte également automatiquement les paragraphes terminaux (feuilles) comme victoires potentielles.
6. **Étape 4b : Visualisation (AffichageGrapheFR).** La classe `AffichageGrapheFR` convertit le graphe interne au format GraphStream, applique l'algorithme de placement force-directed de Fruchterman-Reingold, et affiche une fenêtre interactive.
7. **Sortie :** D'une part, des résultats numériques (chemins trouvés, plus court chemin, probabilité estimée). D'autre part, une visualisation graphique du graphe.

5. Expérimentations et usages

5.1. Cas d'utilisation

5.1.1. Cas 1 : Analyse d'un livre simple

Nous avons testé notre système sur un petit livre d'environ 50 paragraphes. La figure 5.1 montre la visualisation obtenue.

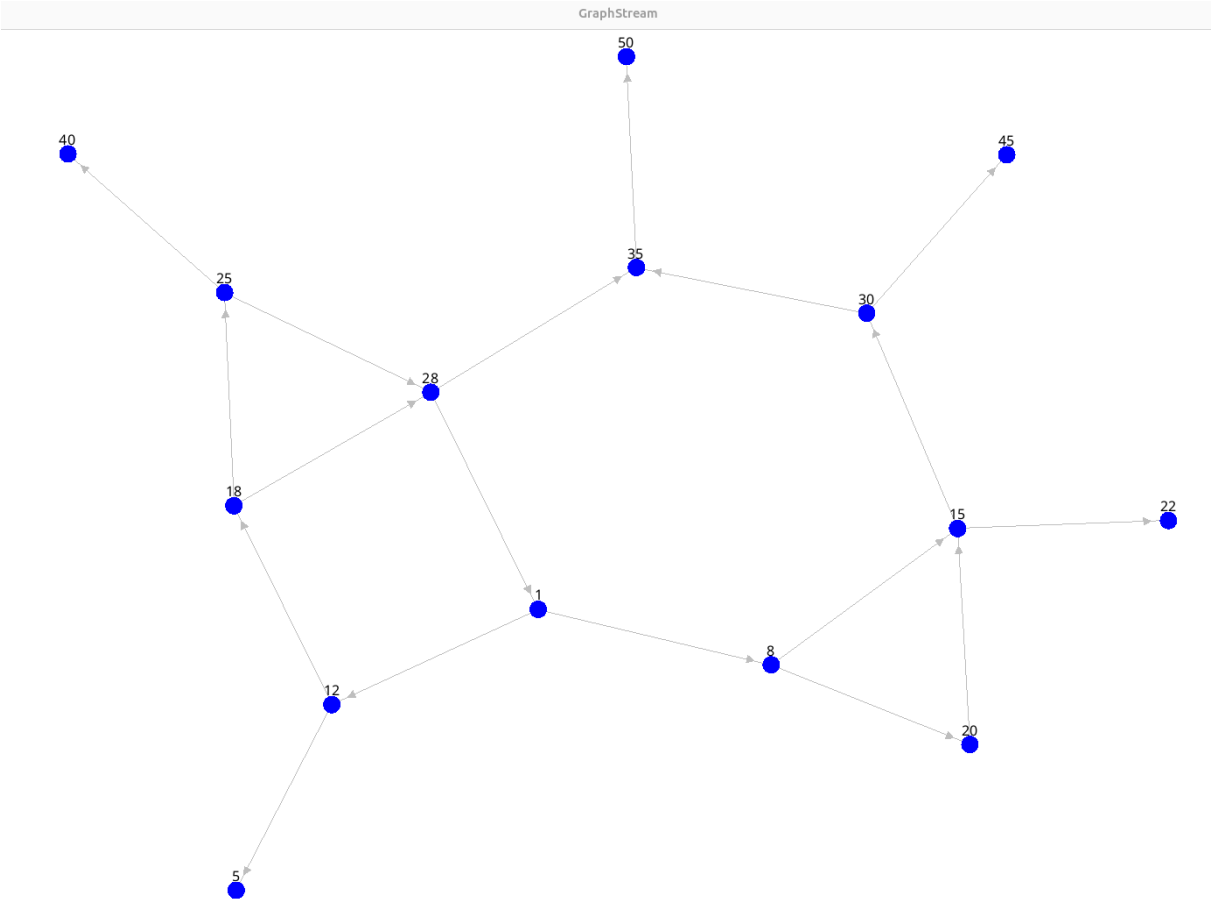

FIGURE 5.1 – Visualisation d'un petit livre

Sur cette visualisation, on distingue clairement le point de départ (en haut à gauche), une structure modérément ramifiée, et plusieurs feuilles correspondant aux fins possibles.

5.1.2. Cas 2 : Calcul du plus court chemin

Pour le même livre, nous avons demandé le plus court chemin entre le paragraphe 1 et une fin victorieuse. Le système a renvoyé le chemin : $1 \to 5 \to 12 \to 18 \to 23$. Ce chemin comporte 4 transitions (5 paragraphes visités). La vérification manuelle a confirmé qu'il n'existe pas de chemin plus court.

5.1.3. Cas 3 : Simulation probabiliste

Nous avons exécuté une simulation de 10 000 parcours aléatoires sur un livre de taille moyenne. La probabilité estimée de victoire était de 0,23 (23%). Ce résultat signifie qu'un lecteur choisissant complètement au hasard n'aurait qu'une chance sur quatre environ de terminer le livre victorieusement. Ce livre peut donc être considéré comme difficile.

5.2. Résultats quantifiables

5.2.1. Temps d'exécution des algorithmes

Nous avons mesuré les temps d'exécution sur trois livres de tailles différentes. Les mesures ont été effectuées sur une machine standard (processeur Intel Core i5, 8 Go RAM).

Livre	Nombre de paragraphes	DFS (exploration)	BFS (chemin)
Petit	52	0,03 s	0,01 s
Moyen	178	0,47 s	0,03 s
Grand	342	2,81 s	0,05 s

Table 5.1 – Temps d'exécution des algorithmes

On observe que le DFS a une complexité qui croît plus vite que le BFS, conformément à l'analyse théorique (exponentielle dans le pire cas). Le BFS reste très rapide même pour les grands livres.

5.2.2. Nombre de chemins trouvés

Pour un livre de 178 paragraphes, l'exploration exhaustive a trouvé 23 chemins victorieux distincts. Le plus court comportait 12 paragraphes, le plus long 47 paragraphes. Cette information permet d'apprécier la richesse du livre : 23 façons différentes de gagner, avec des longueurs variant du simple au quadruple.

5.2.3. Comparaison entre livres

Le tableau 5.2 présente les indicateurs calculés pour trois livres de notre collection.

Indicateur	**Livre A**	**Livre B**	**Livre C**
Nombre de paragraphes	178	342	156
Nombre de choix	245	521	203
Densité ($\times 10^{-3}$)	7,8	4,5	8,4
Degré moyen	1,38	1,52	1,30
Plus court chemin victorieux	12	8	15
Plus long chemin victorieux	47	89	52
Nombre de chemins victorieux	23	187	12
Proportion de victoires	18%	31%	9%
Présence de cycles	Non	Oui	Non

Table 5.2 – Comparaison quantitative de trois livres

5.2.4. Interprétation des résultats

Le Livre B se distingue nettement : c'est le plus volumineux (342 paragraphes), celui qui offre le plus de chemins victorieux (187), la plus forte proportion de victoires (31%), et c'est le seul à contenir des cycles. Ces caractéristiques en font un livre plus complexe et offrant une meilleure rejouabilité.

Le Livre C est le plus court (156 paragraphes) et le plus difficile (seulement 9% de chances de victoire en choix aléatoire). Il propose également le moins de chemins victorieux (12). C'est probablement un livre plus linéaire et plus punitif.

Le Livre A se situe dans une position intermédiaire sur la plupart des indicateurs.

6. Conclusion

Ce projet nous a permis de réaliser un système complet d'analyse de livres dont vous êtes le héros, couvrant toute la chaîne de traitement : extraction automatique de la structure textuelle, construction d'un graphe orienté, exploration des chemins (DFS), calcul du plus court chemin (BFS), simulation probabiliste de la difficulté (Monte Carlo), visualisation interactive via GraphStream, et comparaison quantitative de plusieurs livres.

Au-delà de l'aspect technique, ce projet nous a confortés dans l'idée que les graphes sont un outil fondamental, trop souvent négligé dans les formations informatiques. Les réseaux sociaux, les moteurs de recherche, les applications de navigation — tous ces domaines utilisent intensivement les graphes. Redécouvrir cette structure à travers un projet ludique a été une expérience enrichissante.

Sur le plan théorique, nous avons approfondi notre compréhension des graphes et des algorithmes associés (DFS, BFS). Sur le plan pratique, nous avons renforcé nos compétences en Java, en conception orientée objet et en utilisation de bibliothèques externes. Sur le plan méthodologique, nous avons appris à organiser un travail en équipe et à répartir les tâches.

Plusieurs pistes d'amélioration sont envisageables : l'intégration des conditions d'accès (objets requis, jets de dés), une analyse sémantique du texte, une interface utilisateur enrichie, voire la génération automatique de nouveaux livres.

Ce projet nous a permis d'allier théorie et pratique, et de mesurer la puissance des graphes. Nous espérons que notre travail pourra être utile à d'autres étudiants.